Lập trình mạng · Chương 4

gRPC: streaming, deadline và mã trạng thái

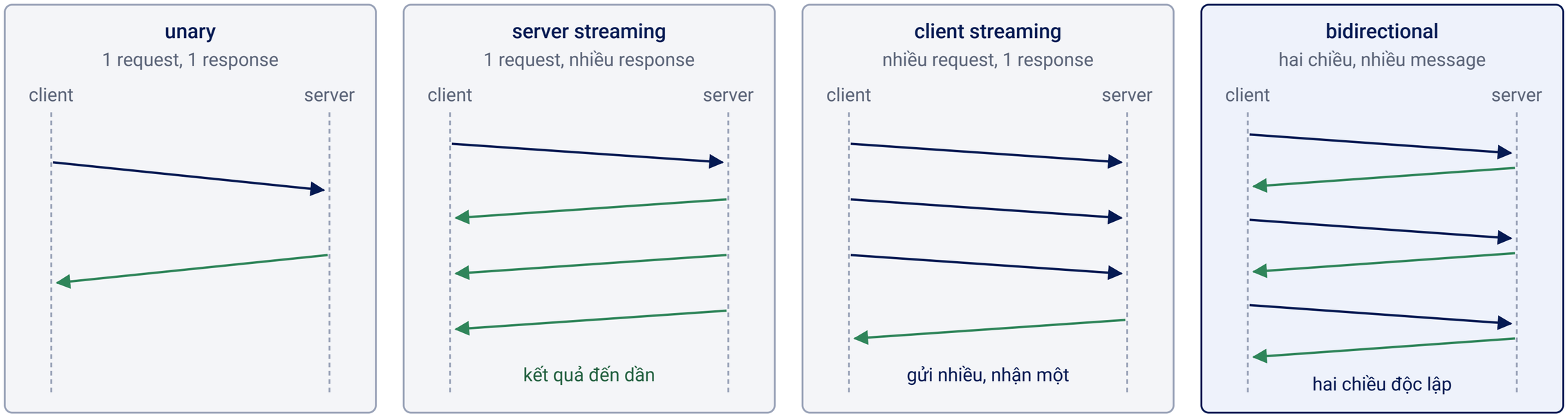
TS. Đặng Ngọc Hùng

Nhiều kết quả, giới hạn thời gian và báo lỗi: ba nhu cầu mới

1. Tra cứu theo tiền tố: hàng nghìn kết quả; với unary, client chờ cho tới khi có **một** message lớn
2. Lời gọi đi qua nhiều dịch vụ: cần **giới hạn thời gian** chung cho cả chuỗi
3. Lời gọi thất bại: cần cách báo **thống nhất** cho client ở mọi ngôn ngữ

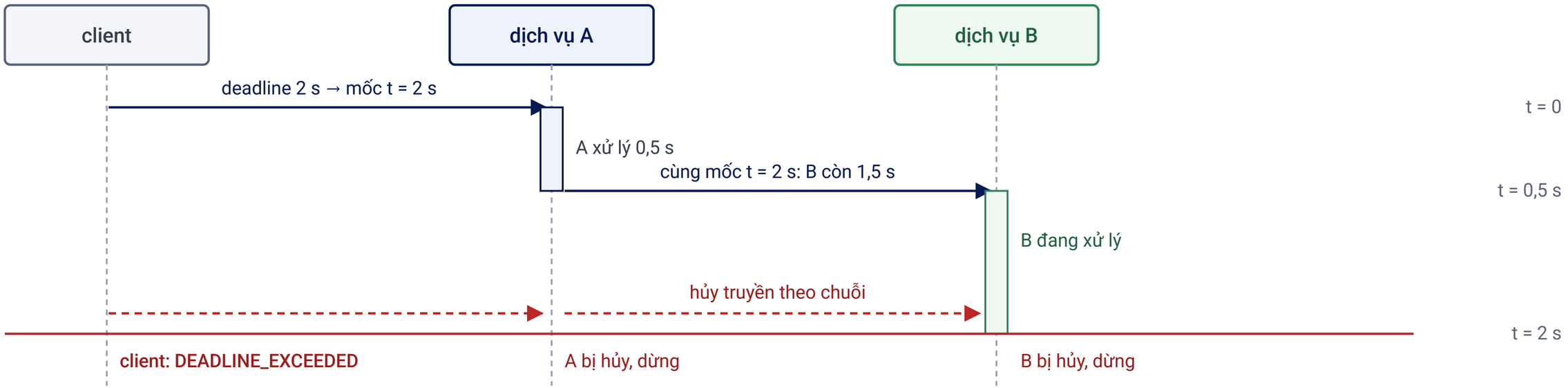
> **Câu hỏi của phần:** gRPC trả nhiều message trên một lời gọi thế nào, đặt deadline ra sao, và báo lỗi bằng gì?

Bốn kiểu RPC: từ khóa stream đánh dấu phía gửi nhiều message



Bốn kiểu RPC: unary, **server streaming**, **client streaming**, **bidirectional streaming**; mỗi lời gọi là một HTTP/2 stream.

Deadline là một mốc thời gian, truyền theo chuỗi lời gọi



Deadline là một mốc thời gian; gRPC mang mốc đó theo các lời gọi tiếp theo và hủy cả chuỗi khi hết giờ (**DEADLINE_EXCEEDED**).

Ba RPC trong một dịch vụ; bidirectional streaming xen kẽ hai chiều

Protobuf

```protobuf
service Lookup {
  rpc Lookup (LookupRequest) returns (LookupReply);                    // unary; không có khóa: mã NOT_FOUND
  rpc Prefix (PrefixRequest) returns (stream Entry);                   // server streaming: trả kết quả dần
  rpc LookupMany (stream LookupRequest) returns (stream LookupReply);  // bidirectional streaming
}
```

Kết quả chạy

```
+   3 ms  sent hanoi
+   7 ms  received hanoi found=true
+ 307 ms  sent atlantis
+ 311 ms  received atlantis found=false
+ 612 ms  sent huế
+ 615 ms  received huế found=true
+ 930 ms  server completed
```

Server streaming: gửi từng mục ngay khi tìm được, client nhận dần

Java

```java
for (Map.Entry<String, String> e : DATA.tailMap(prefix, true).entrySet()) {
    if (!e.getKey().startsWith(prefix)) {
        break;
    }
    sleep(300);  // mô phỏng mỗi mục tốn 300 ms để tìm
    call.onNext(Entry.newBuilder().setKey(e.getKey()).setValue(e.getValue()).build());  // gửi ngay mục này
}
call.onCompleted();
```

Kết quả chạy

```
+ 334 ms  hanoi.hoankiem = Hồ Hoàn Kiếm
+ 631 ms  hanoi.longbien = Cầu Long Biên
+ 938 ms  hanoi.noibai = Sân bay Nội Bài
+1252 ms  hanoi.tayho = Hồ Tây
+1559 ms  hanoi.vanmieu = Văn Miếu – Quốc Tử Giám
+1560 ms  completed
```

Hết deadline: client nhận DEADLINE_EXCEEDED, server biết để dừng

Java

```
stub = stub.withDeadlineAfter(Long.parseLong(args[1]), TimeUnit.MILLISECONDS);  // deadline cho lời gọi
Iterator<Entry> entries = stub.prefix(PrefixRequest.newBuilder().setPrefix(args[0]).build());
} catch (StatusRuntimeException e) {
    System.out.printf("+%4d ms  %s%n", (System.nanoTime() - start) / 1_000_000, e.getStatus().getCode());
```

Kết quả chạy

```
+ 317 ms  hanoi.hoankiem = Hồ Hoàn Kiếm
+ 626 ms  hanoi.longbien = Cầu Long Biên
+ 935 ms  hanoi.noibai = Sân bay Nội Bài
+1011 ms  DEADLINE_EXCEEDED
```

- Server kiểm tra `isCancelled()` trước mỗi lần gửi: log ghi `cancelled by client after 3 entries`

Mã trạng thái báo lỗi thống nhất; luôn đặt deadline

```java
LookupGrpc.newBlockingStub(channel)
        .withDeadlineAfter(2, TimeUnit.SECONDS)  // luôn đặt deadline: gRPC mặc định không có
        .lookup(LookupRequest.newBuilder().setKey(key).build());
} catch (StatusRuntimeException e) {
```

Kết quả chạy

```
port 9199, key atlantis: NOT_FOUND (key atlantis)
port 9198, key hanoi: UNAVAILABLE (io exception)
```

- **Mã trạng thái gRPC**: NOT_FOUND do server chọn, UNAVAILABLE do thư viện báo; client đọc bằng `getStatus()`
- gRPC **mặc định không có deadline**: server treo thì lời gọi chờ không giới hạn

Câu hỏi 1

Tra cứu theo tiền tố có thể trả 10 000 khóa. Nên khai báo RPC này theo kiểu nào?

A. Unary, trả một message chứa cả 10 000 khóa.

B. Client streaming, để client gửi từng khóa.

C. Bidirectional streaming, vì dữ liệu lớn thì bắt buộc dùng hai chiều.

D. Server streaming: server gửi từng khóa ngay khi tìm được.

Câu hỏi 1

Tra cứu theo tiền tố có thể trả 10 000 khóa. Nên khai báo RPC này theo kiểu nào?

A. Unary, trả một message chứa cả 10 000 khóa.

B. Client streaming, để client gửi từng khóa.

C. Bidirectional streaming, vì dữ liệu lớn thì bắt buộc dùng hai chiều.

D. Server streaming: server gửi từng khóa ngay khi tìm được.

Một request, nhiều kết quả: server streaming cho client nhận dần từng khóa. Unary buộc server tìm đủ rồi gửi một message lớn.

Câu hỏi 2

Client hết deadline nên lời gọi bị hủy. Phía server có biết để dừng xử lý không?

A. Có: việc hủy được báo tới server; mã server kiểm tra isCancelled hoặc đăng ký lắng nghe việc hủy để dừng.

B. Không: server luôn chạy hết rồi mới biết.

C. Chỉ biết nếu client gửi thêm một request báo hủy.

D. Mã của server bị dừng ngay lập tức, không cần kiểm tra gì.

Câu hỏi 2

Client hết deadline nên lời gọi bị hủy. Phía server có biết để dừng xử lý không?

A. Có: việc hủy được báo tới server; mã server kiểm tra isCancelled hoặc đăng ký lắng nghe việc hủy để dừng.

B. Không: server luôn chạy hết rồi mới biết.

C. Chỉ biết nếu client gửi thêm một request báo hủy.

D. Mã của server bị dừng ngay lập tức, không cần kiểm tra gì.

gRPC báo việc hủy cho server (ServerCallStreamObserver.isCancelled, Context của lời gọi), nhưng không dừng mã của server giữa chừng: mã phải tự kiểm tra.

Tổng kết

1. **Server streaming, client streaming, bidirectional streaming**: chuỗi message qua **StreamObserver**

2. Deadline là một mốc thời gian, truyền theo chuỗi lời gọi; hết giờ: **DEADLINE_EXCEEDED**, server dừng khi bị hủy

3. **Mã trạng thái gRPC** báo lỗi thống nhất; gRPC mặc định không có deadline

Từ khóa cần nhớ server streaming · client streaming · bidirectional streaming · StreamObserver · mã trạng thái gRPC · DEADLINE_EXCEEDED

Mở rộng

Chủ đề nâng cao

- Interceptor của gRPC (ghi log, xác thực)
- Cân bằng tải phía client

Bài tập mở rộng

Viết RPC Watch(prefix) dạng server streaming: gửi sự kiện mỗi khi một khóa có tiền tố được cập nhật.

Tiếp theo

Message broker: RabbitMQ, exchange, queue và binding